

A SOLVABLE MODEL OF NEAR-CONSTANT-COUNT DATA POISONING

Anonymous authors

Paper under double-blind review

ABSTRACT

Backdoor poisoning of language-model pretraining has been observed to succeed after a near-constant number of poisoned documents, largely independent of the amount of clean data, while packing more poison into each batch makes the attack *less* sample-efficient. Neither finding has a mechanistic explanation. We derive both in closed form. In a solvable model of online training, a trigger that never occurs in clean data ($\rho = 0$) is learned from poison alone, so under SGD the threshold depends on the poison count and not on clean-data size; when the trigger also appears in clean data ($\rho > 0$), a mean-field fixed point recovers the fraction-governed regime of prior theory, and ρ interpolates between the two. A pre-registered test on a transformer trained from scratch confirms this: the poison needed for a trigger that also appears in clean data tracks the predicted count of competing clean uses (123, 346 and 914 examples against a predicted 100, 300 and 900 as training length triples), while a trigger absent from clean data needs fewer than 40 at every length. Under Adam, which is how language models are trained, a second resource appears: because trigger-specific parameters receive gradient only on poisoned steps, each poisoned *batch* moves them by a fixed amount regardless of how many poison examples it contains, and batches closer together than the second-moment memory $1/(1 - \beta_2)$ interfere. This zero-parameter prediction orders attack success across all 11 schedule, length and placement conditions we ran (Spearman 0.67, $p = 0.02$), and a direct causal test confirms it: holding the total poison count fixed and splitting it across more batches raises attack success under AdamW (mean ASR $0.38 \rightarrow 0.84$ at $N = 16$, $p = 0.045$) but not under SGD (flat at 0.00). We also report the theory’s limits: count invariance holds only within a window of training lengths, beyond which attack success decays in a way our account predicts in direction but underestimates in size.

1 INTRODUCTION

Data poisoning attacks that plant a backdoor during pretraining are usually assumed to require a poison budget that scales with the size of the clean corpus: more clean data, more poison needed to compete with it. Souly et al. (2025) overturned this assumption empirically. Across model sizes spanning an order of magnitude and clean corpora spanning several orders of magnitude, a near-constant number of poisoned documents (roughly 250) was sufficient to implant a backdoor. Two further findings concern scheduling. At a fixed per-batch poison density, the frequency of poisoned batches did not change how many poison samples were needed; but at a higher per-batch density, *more* poison samples were needed. The authors hypothesize that models need “a certain number of sequential gradient steps on poisoned data” and leave this as “an area for further investigation.” Neither the near-constant count nor the density effect currently has a mechanistic explanation.

This near-constant-count regime is in apparent tension with an established line of poisoning theory that treats the attack’s success threshold as governed by the *fraction* of poisoned training examples, not their absolute count (Flynn & Granziol, 2025; Flynn et al., 2026), both derived via random-matrix and statistical-physics techniques for high-dimensional linear and generalized linear models. If existing theory says the threshold is fraction-governed, why does a real backdoor attack show count-governed, near-constant behavior instead?

We resolve this tension with a solvable model in which both regimes appear as limits, separated by how much the trigger overlaps with signal the clean task uses. When the trigger never occurs in clean data, clean training exerts no pull on it, and under SGD the attack depends on the poison count alone, independent of clean-data size. When the trigger also occurs in clean data, clean and poison uses compete, and the threshold reverts to the fraction-governed regime of prior theory. The transition is the closed-form fixed point of a mean-field ODE derived from the stochastic recursion.

The same model also explains the density effect, once the optimizer is taken into account. Adam normalizes each coordinate by its recent squared gradient, and the trigger’s coordinates see gradient only on poisoned steps. Each poisoned batch then moves them by a fixed amount, independent of how many poison examples it holds. This makes the “number of sequential gradient steps on poisoned data” conjectured by Souly et al. (2025) precise: under Adam the attack counts poisoned batches, discounted when they fall within the second-moment memory of one another.

Contributions.

1. A closed-form account of why the poison needed for a rare-trigger backdoor does not grow with clean-data size (Theorem 1), and of how it becomes fraction-governed as the trigger’s overlap with clean data grows (Proposition 2), unifying the count-based empirical findings with fraction-based poisoning theory.
2. An exact result for Adam (Proposition 3): the displacement caused by a poisoned batch is independent of the number of poison examples it contains, with an explicit formula for interference between nearby batches. This explains why higher per-batch density makes poisoning less sample-efficient.
3. Validation on a transformer trained from scratch, including a pre-registered test of the ρ transition, and an honest account of where the theory’s reach ends: count invariance holds only within a window of training lengths, and the decay beyond it is predicted in direction but not in magnitude.
4. A causal confirmation of Proposition 3: on the same transformer, holding the total poison count fixed and varying how many batches it is split into raises attack success under AdamW but not under SGD (Section 7.3), isolating the optimizer as the cause rather than the schedule alone.

2 RELATED WORK

Empirical poisoning at pretraining scale. Souly et al. (2025) is the paper this work explains: they report near-constant poison counts across model scale and explicitly state they lack a mechanism. Backdoor attacks that tie a trigger to a target behaviour were introduced by Gu et al. (2017). Carlini et al. (2023) show that poisoning web-scale training datasets is practical, and Wan et al. (2023) show that as few as 100 poison examples can manipulate instruction-tuned language models. None of these works characterizes the count-versus-fraction distinction or its dependence on trigger rarity.

Fraction-governed poisoning theory. Flynn & Granziol (2025) derive a closed-form poisoning threshold for ridge regression via random matrix theory, parameterizing the attack by the *fraction* θ of one class relabeled along a fixed direction. Flynn et al. (2026) study trigger *strength* at a fixed poison ratio in a high-dimensional generalized linear model and find a non-monotonic success curve as strength varies. Both implicitly study directions with $O(1)$ overlap with the ambient signal – our $\rho = O(1)$ regime – and neither addresses a count-governed limit, scheduling invariance, or dataset-size invariance. We recover their scaling form as the dense/generic-trigger limit of our model (Section 5) rather than treating it as a separate phenomenon.

Theoretical tools. Our derivation uses a mean-field/stochastic-approximation reduction of a discrete SGD recursion to an autonomous ODE, in the spirit of classical stochastic approximation theory (Robbins & Monro, 1951; Benveniste et al., 1990). To our knowledge this reduction has not previously been applied to backdoor/poisoning attack theory; a targeted literature search for this combination of techniques and problem domain returned no close prior work.

Adaptive optimizers and rare features. Adam (Kingma & Ba, 2015) and its decoupled-weight-decay variant AdamW (Loshchilov & Hutter, 2019) normalize each coordinate by a running second-moment estimate. Kunstner et al. (2024) show that under heavy-tailed class imbalance gradient

descent makes slow progress on low-frequency classes while Adam does not, which helps explain Adam’s advantage on language models. We study the same normalization from the attacker’s side: for a trigger that appears only in poison data, it makes each poisoned batch move the trigger’s parameters by a fixed amount (Proposition 3). We are not aware of prior work connecting optimizer dynamics to poison-count or backdoor scaling.

3 PROBLEM SETUP

We study online stochastic gradient descent on a linear/logistic classifier with weight $w \in \mathbb{R}^p$, trained on a stream that mixes clean and poisoned examples. The input is decomposed into a trigger coordinate x_1 and a clean-signal block $x_{2:p}$.

- **Poisoned examples** set $x_1 = c$ for a fixed constant c and assign the attacker’s target label, regardless of the true clean label.
- **Clean examples** draw x_1 from a distribution correlated with the clean classification task with overlap strength $\rho \in [0, 1]$: at $\rho = 0$, x_1 carries no information about the clean task (a *rare* trigger); at $\rho > 0$, x_1 is itself a genuine, if partial, signal feature for the clean task (a *generic* trigger).

Let $z = c \cdot w_1$ denote the model’s margin along the trigger direction. The attack succeeds once z exceeds the threshold set by the decision rule. Our goal is to characterize, as a function of ρ , the poison count N^* needed to reach a fixed target margin, and how N^* depends on the total number of clean training steps n_{clean} and on how poison examples are scheduled across training.

4 THEOREM 1: THE COUNT-GOVERNED REGIME ($\rho = 0$)

Claim. Under SGD, when x_1 is exactly uncorrelated with the clean label, w_1 is updated only on poison-containing steps. Consequently, in this model, (a) z depends on the total poison mass s but, in the small-step limit, not on how it is divided among batches; and (b) z does not depend on the number of clean gradient steps, since clean steps contribute zero expected update to w_1 . Section 6 shows how (a) changes under Adam.

Theorem 1. *The per-step update $z_t = z_{t-1} + \eta c^2 \kappa_t \sigma(-z_{t-1})$ (where $\kappa_t \geq 0$ is the poison mass at step t , zero on clean steps, and σ is the logistic function), in the continuum limit $\eta c^2 \kappa_t \ll 1$ (many small poison-touching steps), satisfies the closed form*

$$z(s) + e^{z(s)} = 1 + \eta c^2 s, \quad (1)$$

where $s = \sum_{t' \leq t} \kappa_{t'}$ is cumulative poison mass. This yields a crossover scale $s^* = O(1/\eta c^2)$: for $s \ll s^*$, $z \approx \eta c^2 s/2$ (linear growth); for $s \gg s^*$, $z \approx \log(\eta c^2 s)$ (logarithmic, saturating growth) – an $O(1)$, dataset-size-independent poison budget beyond which additional poison buys only diminishing returns.

Proof. Separating variables in $dz/ds = \eta c^2 \sigma(-z)$ gives $(1 + e^z) dz = \eta c^2 ds$; integrating both sides ($\int (1 + e^z) dz = z + e^z$) and fixing the constant at $z(0) = 0$ gives Equation 1 directly. $\square$

Regime of validity. The continuum approximation is accurate when each step’s perturbation is small relative to the local curvature of $\sigma(-z)$. Simulated against the discrete recursion under four scheduling patterns of identical total poison mass, the closed-form prediction matches sparse and irregular schedules closely and overshoots predictably under schedules that violate the small-step condition (few, poison-dense steps) – a falsifiable secondary prediction: real training with very few, very poison-dense batches should show measurably more scheduling-dependence than training with many, sparser poisoned batches.

5 PROPOSITION 2: THE FRACTION-GOVERNED REGIME ($\rho > 0$)

When x_1 is a genuine, if partial, clean-task feature, every clean step also updates w_1 , pulling it toward correct classification with strength $\propto \rho^2$, opposing the poison-driven push, at rate $\lambda = n_{\text{poison}}/n_{\text{clean}}$ among clean steps.

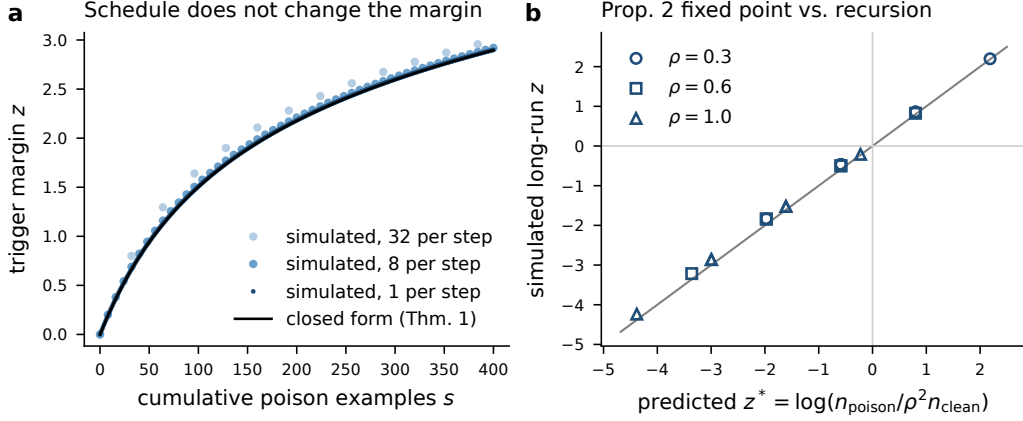


Figure 1: Both closed forms against the exact discrete recursion. (a) Theorem 1 ($\rho = 0, \eta c^2 = 0.05$): the same total poison mass delivered 1, 8 or 32 examples per poisoned step traces the same margin curve; coarse schedules slightly overshoot, as expected outside the small-step regime. (b) Proposition 2 ($\eta = 0.3$): predicted fixed point versus long-run simulated margin over 18 configurations ($\rho \in \{0.3, 0.6, 1.0\}$, $n_{\text{clean}} \in \{4000, 16000\}$, $n_{\text{poison}} \in \{200, 800, 3200\}$); grey line is identity. Maximum deviation 0.145, sign agreement in all 18.

Proposition 2. In the mean-field continuum limit, $dz/dt = \eta c^2 \sigma(-z) \lambda - \eta \rho^2 \sigma(z)$. Its fixed point is

$$z^* = \log\left(\frac{c^2 \lambda}{\rho^2}\right) = \log\left(\frac{c^2 n_{\text{poison}}}{\rho^2 n_{\text{clean}}}\right), \quad (2)$$

so the attack succeeds ($z^* > 0$) if and only if $n_{\text{poison}} > (\rho^2/c^2) n_{\text{clean}}$ – an exact $N^* \propto \rho^2 n_{\text{clean}}$ threshold.

Proof. Using $\sigma(-z) = 1 - \sigma(z)$, the fixed-point condition $c^2 \lambda \sigma(-z^*) = \rho^2 \sigma(z^*)$ rearranges to $\sigma(z^*) = c^2 \lambda / (\rho^2 + c^2 \lambda)$, and solving $e^{z^*} / (1 + e^{z^*})$ for this value gives $e^{z^*} \rho^2 = c^2 \lambda$, i.e. Equation 2. $\square$

Verification. Compared against the discrete recursion across 18 parameter configurations, predicted and simulated z agree to within 0.1–0.15 in every case, always the correct sign and qualitative regime. This residual shrinks linearly in η (verified across $\eta \in \{0.3, 0.1, 0.03, 0.01, 0.003\}$, $\text{offset}/\eta \approx 0.27$ – 0.29 throughout) – the standard signature of first-order Euler discretization bias in a stochastic-approximation limit, confirming Equation 2 as the correct $\eta \rightarrow 0$ limit rather than an artifact.

6 PROPOSITION 3: ADAPTIVE OPTIMIZERS COUNT POISON EVENTS

Theorem 1 assumes SGD, under which a poisoned step moves the trigger weight in proportion to the number of poison examples it contains. Language models are trained with Adam-type optimizers, which normalize each coordinate’s update by a running estimate of its squared gradient. At $\rho = 0$ the trigger’s parameters receive gradient *only* on poisoned steps, which is exactly the regime in which this normalization matters most. Consider one such coordinate θ under Adam with momentum β_1 , second-moment decay β_2 , learning rate η and $\beta_1 < \sqrt{\beta_2}$ (as for the standard 0.9, 0.999). A *poison event* is a step whose batch contains at least one poison example, giving gradient $g_n \neq 0$; all other steps give zero gradient to θ .

Proposition 3. Let E poison events occur with spacing $\Delta \gg 1/(1 - \beta_1)$ and equal gradient magnitude $|g_n| = g$ (the small-margin regime), and neglect bias correction and weight decay. Then

the total displacement of θ is

$$z_E = D_0 \sqrt{1 - \beta_2^\Delta} \sum_{n=1}^E (1 - \beta_2^{n\Delta})^{-1/2}, \quad D_0 = \frac{\eta(1 - \beta_1)}{(1 - \beta_1/\sqrt{\beta_2})\sqrt{1 - \beta_2}}, \quad (3)$$

which does not depend on g , and hence not on how many poison examples each event contains. In particular, $z_E \rightarrow E D_0$ when $\Delta \gg 1/(1 - \beta_2)$ (every event counts fully), and $z_E \approx 2D_0\sqrt{E}$ when $\Delta \ll 1/(1 - \beta_2)$ (clustered events count only as $\sqrt{E}$).

Proof. Because $\Delta \gg 1/(1 - \beta_1)$, the first moment has decayed to zero before each event. Let u_n be the second moment immediately after the n -th event. After an event with gradient g , the moments evolve as $m_j = \beta_1^j(1 - \beta_1)g$ and $v_j = \beta_2^j u_n$, so the event displaces θ by $\eta \sum_{j \geq 0} m_j / \sqrt{v_j} = \eta(1 - \beta_1)g u_n^{-1/2} \sum_{j \geq 0} (\beta_1/\sqrt{\beta_2})^j = \eta(1 - \beta_1)g / ((1 - \beta_1/\sqrt{\beta_2})\sqrt{u_n})$. The second moment obeys $u_n = \beta_2^\Delta u_{n-1} + (1 - \beta_2)g^2$ with $u_0 = 0$, so $u_n = (1 - \beta_2)g^2(1 - \beta_2^{n\Delta})/(1 - \beta_2^\Delta)$. Substituting and summing over n gives Equation 3; g cancels. The two limits follow from $\beta_2^\Delta \rightarrow 0$, and from $1 - \beta_2^\Delta \approx x(1 - \beta_2)$ together with $\sum_{n \leq E} n^{-1/2} \approx 2\sqrt{E}$. $\square$

Verification. Simulating the Adam recursion directly, a single event moves θ by exactly 31.8η whether it carries 1, 4, 16 or 64 poison examples, matching D_0 at $(\beta_1, \beta_2) = (0.9, 0.999)$. Across spacings from 200 to 10,000 steps and 4 to 16 events, Equation 3 matches the simulated displacement to within 0.01%; the error is 0.07% at spacing 50 and 0.8% at spacing 20, where successive momentum tails begin to overlap (Appendix A, Table 2). The proposition assumes equal gradient magnitude $|g_n| = g$ across events, which holds exactly in this idealization because every poison example sets the trigger input to the same constant c ; it need not hold once the trigger’s weight θ itself has moved enough to change $\sigma(-\theta)$ between events, which is why Table 3 simulates each condition’s own schedule rather than applying Equation 3 with a single g .

Consequence: two channels. Proposition 3 applies to parameters that are specific to the trigger, such as its embedding, which see gradient only on poisoned steps. Parameters shared with the clean task, such as the output layer, have their second-moment estimate set by clean gradients; for them Adam acts like SGD with a rescaled step, and a poisoned batch moves them in proportion to the number of poison examples it contains. Under Adam, the attack therefore draws on two resources: poisoned batches, through the trigger-specific channel (discounted when batches fall within $1/(1 - \beta_2)$ steps of one another), and poison examples, through the shared channel. Under SGD both channels count examples. The resulting prediction, which SGD does not make, is that at a fixed total number of poison examples, spreading them over more batches increases attack success, and clustering batches in time reduces it. Section 7 tests this directly by comparing the two optimizers. As a first check of the mechanism, we simulate the scalar Adam recursion above under the exact poison schedule of each transformer condition in Section 7 – using that experiment’s T , poison count and batch spacing, with AdamW’s weight decay and bias correction and no fitted parameter – and rank the resulting displacements against the measured mean ASR. Over the 11 conditions the Spearman rank correlation is 0.67 ($p = 0.023$; Appendix A, Table 3).

7 REAL-TRANSFORMER VALIDATION

We test the theory’s predictions on a transformer trained from scratch, sweeping the axes studied by Souly et al. (2025): how poison is scheduled, how long clean training runs, and whether the trigger also occurs in clean data.

7.1 SETUP

Task and model. Clean sequences of length 32 are drawn from a fixed bigram language model over a 64-token vocabulary: the first token is uniform, and each subsequent token is sampled from $\text{softmax}(B_{x_{t-1}, :})$ with $B_{ij} \sim \mathcal{N}(0, 1)$ drawn once and shared by all runs. The model is a causally-masked transformer (Vaswani et al., 2017) with 4 layers, width 128, 4 heads, feed-forward width 512 and learned positional embeddings (0.81M parameters), trained with next-token cross-entropy and AdamW (Loshchilov & Hutter, 2019) (learning rate 10^{-3} , weight decay 0.01, batch 64). Every

step draws a fresh batch, so no clean sequence is repeated; we report training length T in steps. Each run uses a fresh random initialization.

Poisoning. The trigger is a dedicated token (id 64) that the bigram model never produces. A poison example is a clean sequence with the trigger written at position 16 and the attacker’s target token at position 17; poison examples replace clean sequences within a batch. A schedule is specified by the total poison count N and the number of poison examples per poisoned batch (the *density*); poisoned batches are spaced evenly from step 5 to step $T - 5$.

Attack success rate (ASR). After training, we place the trigger at position 16 of 200 fresh uniformly random sequences and report the fraction for which the model’s most likely next token is the target.

Rare versus generic trigger. In the rare condition the trigger never appears in clean data ($\rho = 0$). In the generic condition a fraction of *clean* sequences contain the same trigger in the same slot followed by a fixed benign token, at an average of 0.1 occurrences per step. This mirrors the poison exactly except for the continuation, so clean and poison signals compete for the same prediction, which is the situation Proposition 2 models. Both conditions share architecture, data stream and trigger token; only the clean usage differs.

7.2 RESULTS

Training length (Figure 2a). With $N = 16$ poison examples at density 4, ASR is not constant in training length. Pooling two independent sweeps (20 runs per length up to 32,000; 6 at 64,000), mean ASR is 0.10 at $T = 500$, 0.52 at 2,000, 0.49 at 8,000, 0.18 at 32,000 and 0.13 at 64,000. Both the rise (500 versus 2,000–8,000; one-sided Mann–Whitney $p = 7 \times 10^{-5}$) and the decline (32,000–64,000 versus 2,000–8,000; $p = 6 \times 10^{-4}$) are significant; the two sweeps agree on this shape but not on whether the peak lies at 2,000 or 8,000. Individual runs are close to bimodal, ending near 0 or near 1. Theorem 1’s invariance therefore holds only approximately, within a window of training lengths. The decline is not forgetting after the final dose, because the last poisoned batch is always 5 steps before the end of training. Instead, the spacing between the four poisoned batches grows as $T/3$, consistent with each dose partially decaying before the next arrives. The Adam account of Section 6 reproduces this shape once decoupled weight decay is included: simulating the trigger coordinate under each experiment’s own schedule gives a displacement whose maximum falls at $T = 8,000$, inside the observed peak region, while setting the weight decay to zero makes it rise monotonically through $T = 64,000$ (Appendix A). The predicted decline is far smaller than the observed one (17% against 73% from $T = 8,000$ to 64,000), so decoupled weight decay accounts for the turning point and the direction but not the magnitude. As a consistency check, a three-parameter rise-times-decay curve, $A(1 - e^{-T/\tau_1})e^{-T/\tau_2}$, fitted to $T \leq 32,000$ predicts 0.04 at $T = 64,000$ (bootstrap 95% interval $[0, 0.20]$), against the observed 0.13 ± 0.07 : consistent, but with too few runs at 64,000 to count as a confirmation.

Scheduling (Figure 2b). At fixed $N = 16$ and $T = 3,000$, spreading poison over 16 batches gives mean ASR 0.53, 4 batches give 0.23, and a single batch gives 0 in all 5 seeds. The 16- and 4-batch schedules both end at step 2,995, so their difference is not a placement effect; the single-batch schedule necessarily sits at step 5 and is confounded with early placement. The direction of the effect is opposite to the scalar model, where dense steps slightly *overshoot* (Figure 1a): in the transformer, how many separate times the trigger is reinforced matters in addition to the total count.

Rare versus generic trigger. Proposition 2 predicts that the poison count needed to flip the prediction grows in proportion to the number of competing clean uses, while the rare trigger’s threshold does not depend on training length. With poison and clean uses of the trigger entering symmetrically ($c = \rho$), $z^* > 0$ exactly when poison examples outnumber competing clean uses; at 0.1 clean uses per step this predicts thresholds near 100, 300 and 900 poison examples at $T = 1,000$, 3,000 and 9,000; we fixed this prediction before running the sweep ($N \in \{25, 75, 225, 675, 2025\}$, density 8, 3 seeds per cell). The result matches (Figure 3). For the generic trigger, the poison count needed for 50% ASR, interpolated log-linearly between grid points, is 123, 346 and 914 at $T = 1,000$, 3,000 and 9,000 (bootstrap 95% intervals over seeds $[116, 130]$, $[239, 383]$ and $[408, 1174]$), or 1.23, 1.15 and 1.02 times the expected number of clean uses of the trigger. The fitted log–log slope against T is 0.91 (95% interval $[0.54, 1.04]$; predicted 1). For the rare trigger, mean ASR at the smallest budget of 25 poison examples is already 0.67 and 0.83 at $T = 1,000$ and 3,000, so the threshold lies below the grid; at $T = 9,000$ it is 39 (interval $[33, 42]$), 23 times lower than for the generic trigger at the

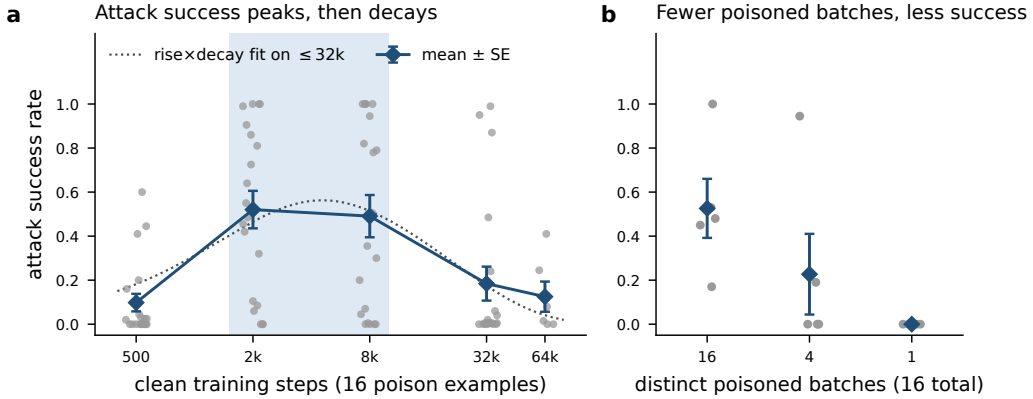


Figure 2: Transformer validation, rare trigger. (a) ASR versus training length at $N = 16$ poison examples, density 4. Grey points are individual runs (20, 20, 20, 20, 6; two independent sweeps pooled); diamonds are means $\pm$ standard error; the dotted curve is a rise-times-decay fit to $T \leq 32,000$ only, extrapolated to 64,000; the shaded band marks the peak region. (b) ASR versus the number of distinct poisoned batches at $N = 16$, $T = 3,000$ (5 seeds). The single-batch schedule sits at step 5 and is confounded with early placement.

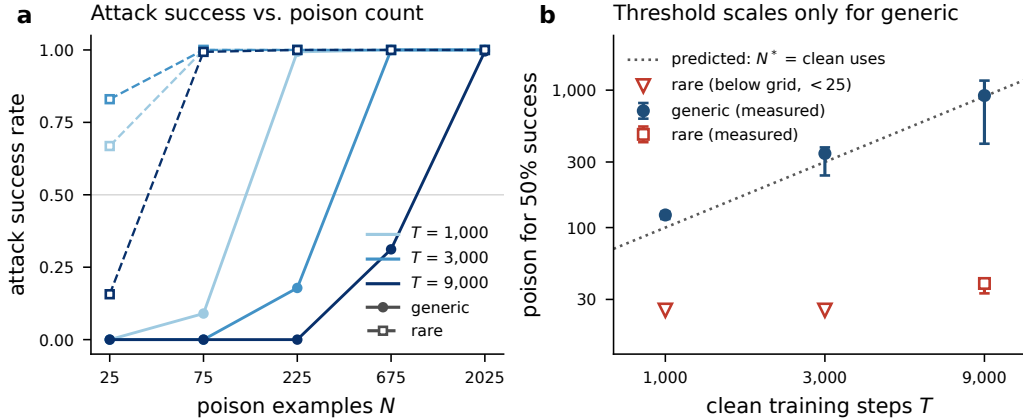


Figure 3: Rare versus generic trigger (pre-registered test of Proposition 2). (a) Mean ASR over 3 seeds versus poison count; solid lines are the generic trigger (0.1 clean uses per step), dashed lines the rare trigger, and colour marks training length. (b) Poison count for 50% ASR. The dotted line is the prediction fixed before the run, N^* equal to the number of clean uses; error bars are bootstrap 95% intervals over seeds. Triangles mark rare-trigger thresholds below the smallest budget tested (25).

same length. The rare threshold rises slightly at the longest length, in line with the training-length decline in Figure 2a, but grows far less than the ninefold increase in clean data.

7.3 CAUSAL TEST OF PROPOSITION 3: ADAMW VERSUS SGD

Section 7’s conditions vary the optimizer’s real schedule, but not the optimizer itself, so the batch-count effect there is a correlation. To test it causally, we train the same transformer with SGD (momentum 0.9, learning rate 0.1, tuned to match AdamW’s clean-data loss) instead of AdamW, and hold the total poison count N fixed while varying how many batches it is split into, at $T = 3,000$ (5 seeds per cell). Proposition 3 predicts more batches raise attack success under AdamW and not under SGD.

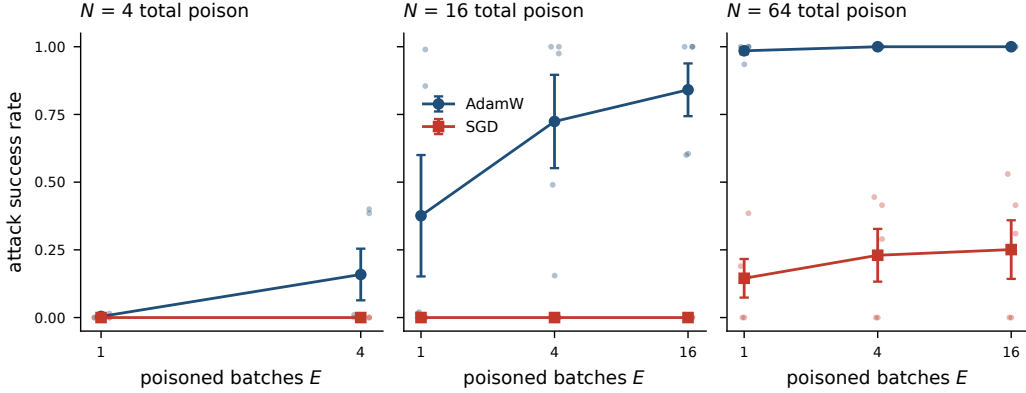


Figure 4: Attack success versus the number of poisoned batches E at three fixed poison totals N , comparing AdamW against SGD (momentum 0.9, lr 0.1) on the same transformer, $T = 3,000$, 5 seeds per point. Small points are individual runs; large markers are means $\pm$ standard error. Spreading a fixed poison budget over more batches raises attack success under AdamW but not under SGD, as Proposition 3 predicts; both optimizers are near floor at $N = 4$ and near ceiling at $N = 64$ –256, where no separation is possible.

The prediction holds (Figure 4). At $N = 16$, mean ASR under AdamW rises from 0.38 at 1 batch to 0.84 at 16 batches (one-sided Mann–Whitney $p = 0.045$); at the same N under SGD, mean ASR is 0.00 at every batch count. At $N = 64$, AdamW is already saturated (0.98–1.00) and SGD shows a small, non-significant rise (0.14 $\rightarrow$ 0.25, $p = 0.26$) – far smaller than AdamW’s effect at $N = 16$, and consistent with the shared-parameter channel of Section 6 contributing under both optimizers while the trigger-specific channel contributes only under AdamW. At $N = 4$ and $N = 256$, both optimizers are too close to floor or ceiling respectively to separate.

8 DISCUSSION AND LIMITATIONS

What the theory explains. The trigger’s overlap with clean data decides whether the poison budget is set by a count or by a fraction of the data (Theorem 1, Proposition 2). Under Adam, trigger-specific parameters additionally count poisoned batches (Proposition 3), which accounts for the observation of Souly et al. (2025) that denser poisoned batches need more poison samples, and for our finding that spreading a fixed budget over more batches raises attack success.

What it explains only partly. Attack success on the transformer declines for training runs much longer than the poison schedule (Figure 2a). This is not forgetting after the final poisoned batch, and placing the poison early or late did not change attack success significantly ($p = 0.54$, 5 runs each). The Adam account does reproduce the shape: with decoupled weight decay its maximum falls at $T = 8,000$, within the observed peak region of 2,000–8,000 (whose two means, 0.52 and 0.49, we cannot separate), and it declines thereafter; with the weight decay removed the decline disappears entirely. What it does not reproduce is the size of the decline, 17% predicted against 73% observed. Souly et al. (2025) likewise report that attack success decays gradually under continued clean training. Closing this gap plausibly requires modelling the drift of parameters shared with the clean task, which the scalar reduction does not track.

Scope. All experiments use a synthetic bigram task and a 0.81M-parameter transformer, following the scope of prior solvable-model poisoning theory (Flynn & Granziol, 2025; Flynn et al., 2026). The theory is a scalar reduction: it tracks the margin along the trigger direction and does not model interactions between coordinates, which a full high-dimensional treatment would capture. Individual runs end close to either 0 or 1 attack success, so means carry large standard errors. Validating the Adam prediction on a pretrained model with a natural-language corpus is the most direct next step.

9 CONCLUSION

Whether a backdoor attack’s cost is set by a count or by a fraction of the training data is decided by a single property of the trigger: how often it already occurs in clean text. A trigger the clean corpus never produces is learned from poison alone, so no amount of additional clean data raises its price – which is why a near-constant number of poisoned documents suffices in practice. A trigger that clean text also uses must outcompete those uses, and its price grows with the corpus. The same model, applied to Adam rather than SGD, explains why packing poison more densely into batches wastes it: the trigger’s own parameters are updated only on poisoned steps, so Adam’s per-coordinate normalization makes each poisoned batch worth a fixed amount however much poison it carries.

For defenders this reframes what to measure. Corpus frequency of a candidate trigger, not poison fraction, predicts how expensive the attack is, so auditing effort is best spent on rare tokens with unusual continuations – exactly the regime in which scaling the clean corpus buys no protection at all. The clearest open problem is the decay we report but cannot fully explain: backdoors implanted early erode over long clean training faster than our account predicts, and quantifying that erosion would turn a qualitative reassurance into a usable bound.

AI USE STATEMENT

We used generative AI tools for tasks with required disclosure under ICLR’s AI Policy for Authors: the initial research idea, the literature novelty screening that shaped and pruned it, the theoretical derivations in Sections 4, 5 and 6, all code implementing the experiments in Section 7, and the drafting of this manuscript’s text were produced by an AI agent (a large language model operating with tool access to code execution and literature search). A second, smaller language model was used for adversarial review of the derivations in Sections 4 and 5 – the specific issues it raised and how each was resolved are recorded in the supplementary theory document. We have not used generative AI tools to fabricate, alter, or select experimental results reported in this paper; every reported number is the direct output of the code in the supplementary material, run on real hardware, with no post-hoc filtering of runs or seeds. A human provisioned and operated the GPU compute the agent’s code ran on and relayed raw output back to the agent verbatim, but did not independently derive, implement, or verify the technical content. We have reviewed all AI-assisted work: every derivation was checked against a discrete-recursion simulation before being reported as verified (Sections 4–6), every quotation and bibliographic entry was checked against the cited source, and every experimental anomaly encountered was investigated for implementation bugs before being reported as a real effect (Section 7). We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI. Note: the full text of the referenced ICLR 2027 AI Policy for Authors document was not available to us at drafting time, so we were unable to cross-check this statement’s disclosure categories against it directly; this statement follows the structure requested in the conference style-file template as closely as possible in its absence.

ETHICS STATEMENT

This work analyzes a known backdoor-poisoning vulnerability and introduces no new attack. The finding that spreading poison over more batches increases its effect under Adam is relevant to attackers, but an attacker who contributes documents to a web corpus typically does not control how they are batched; the same finding is relevant to defenders, since it indicates which data-ordering choices change a model’s exposure and that auditing for rare tokens with unusual continuations targets the regime in which a constant poison count suffices. All experiments use a synthetic task and small models trained from scratch; no deployed model was attacked.

REPRODUCIBILITY STATEMENT

Section 7.1 specifies the task, model, poisoning procedure and evaluation. The supplementary code contains a single script that runs every experiment (schedule, training length, placement, rare versus generic trigger, and AdamW versus SGD) from command-line arguments, together with the exact commands and the raw per-run outputs used for every number and figure. For the schedule, training length, placement and trigger experiments, the seed fixes the clean data stream and the poison

schedule but not the model initialization, so we treat each run as an independent initialization; the AdamW-versus-SGD experiment also seeds the initialization. All runs are reported, with no filtering. Full proofs are given in Sections 4–6, and the scripts that check each closed form against its discrete recursion are included in the supplementary material.

REFERENCES

- Albert Benveniste, Michel Métivier, and Pierre Priouret. *Adaptive Algorithms and Stochastic Approximations*. Springer, 1990.
- Nicholas Carlini, Matthew Jagielski, Christopher A. Choquette-Choo, Daniel Paleka, Will Pearce, Hyrum Anderson, Andreas Terzis, Kurt Thomas, and Florian Tramèr. Poisoning web-scale training datasets is practical. *arXiv preprint arXiv:2302.10149*, 2023.
- Donald Flynn and Diego Granziol. A linear approach to data poisoning. *arXiv preprint arXiv:2505.15175*, 2025.
- Donald Flynn, Hadas Yaron Goldhirsh, Jonathan P. Keating, and Inbar Seroussi. When stronger triggers backfire: A high-dimensional theory of backdoor attacks. *arXiv preprint arXiv:2605.22481*, 2026.
- Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. BadNets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*, 2017.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- Frederik Kunstner, Robin Yadav, Alan Milligan, Mark Schmidt, and Alberto Bietti. Heavy-tailed class imbalance and why Adam outperforms gradient descent on language models. *arXiv preprint arXiv:2402.19449*, 2024.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- Alexandra Souly, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mougán, Vasilios Mavroudis, Erik Jones, Chris Hicks, Nicholas Carlini, Yarin Gal, and Robert Kirk. Poisoning attacks on LLMs require a near-constant number of poison samples. *arXiv preprint arXiv:2510.07192*, 2025.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Alexander Wan, Eric Wallace, Sheng Shen, and Dan Klein. Poisoning language models during instruction tuning. *arXiv preprint arXiv:2305.00944*, 2023.

A ADDITIONAL VERIFICATION OF PROPOSITION 3

Table 2 compares Equation 3 with a direct simulation of the Adam recursion ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$, no weight decay, bias correction negligible) for a coordinate that receives one poison event every Δ steps. Table 3 lists, for each transformer condition in Section 7, the displacement predicted by simulating the same recursion with the experiment’s actual poison schedule (with AdamW weight decay 0.01 and bias correction), next to the observed mean attack success rate. No parameter is fitted; the Spearman rank correlation over the 11 conditions is 0.67 ($p = 0.023$). Repeating the training-length rows with the weight decay set to zero gives displacements of 0.030, 0.052, 0.078, 0.082 and 0.082 at $T = 500$ to 64,000: monotonically increasing, with no peak. The non-monotonicity in Table 3 is therefore produced by decoupled weight decay acting over the poison-free remainder of training.

Table 1: Predicted trigger displacement at each training length, with and without decoupled weight decay

Training length T	Displacement (weight decay 0.01)	Displacement (weight decay 0)
500	0.0296	0.0297
2,000	0.0518	0.0522
8,000	0.0751	0.0778
32,000	0.0709	0.0816
64,000	0.0621	0.0816

Table 2: Closed form (Equation 3) versus simulated Adam displacement

Events E	Spacing Δ	Simulated	Equation 3	Rel. error (%)
16	50	0.2267	0.2268	0.07
16	200	0.2712	0.2712	0.00
8	500	0.1825	0.1825	0.00
4	1,000	0.1103	0.1103	0.00
16	2,000	0.4752	0.4752	0.00
4	10,000	0.1271	0.1271	0.00
64	20	0.5132	0.5175	0.82

Table 3: Zero-parameter Adam prediction versus observed attack success on the transformer (schedule rows give the number of poisoned batches)

Condition	Predicted displacement	Observed mean ASR
Training length: $T = 500$	0.030	0.10
Training length: $T = 2,000$	0.052	0.52
Training length: $T = 8,000$	0.075	0.49
Training length: $T = 32,000$	0.071	0.18
Training length: $T = 64,000$	0.062	0.12
Schedule: 16	0.185	0.53
Schedule: 4	0.060	0.23
Schedule: 1	0.005	0.00
Placement: early	0.149	0.13
Placement: even	0.371	0.55
Placement: late	0.257	0.29